

A
PROJECT REPORT
ON
**Applied Machine Learning:
Dual-Pipeline Regression & Classification**
BACHELOR OF TECHNOLOGY
IN
SCHOOL OF COMPUTER SCIENCE



B.TECH. - Semester IV

Jan - May 2026

Under the guidance of

Prof. Shahinur Lashkar
Assistant Professor
SOCS, UPES

Submitted by

Yashasvi Solanki
SAP ID: 590012593
Batch: AIML

Contents

1	Problem Statement & Objectives	2
1.1	Problem Statement	2
1.2	Detailed Project Objectives	2
2	Dataset Details	2
2.1	Dataset 1: Diamonds (Regression Task)	2
2.2	Dataset 2: Water Potability (Classification Task)	3
3	Data Preprocessing & Feature Engineering	3
3.1	Categorical Encoding (One-Hot Encoding)	3
3.2	Handling Missing Data & Feature Scaling	3
3.3	Anti-Leakage Splitting & SMOTE	4
4	Models Used & Training Architecture	4
4.1	Regression Pipeline Models	4
4.2	Classification Pipeline Models	5
5	Comparative Analysis & Visualizations	5
5.1	Regression Pipeline Results (Diamonds)	6
5.2	Classification Pipeline Results (Water Potability)	7
6	Observations & Insights	10
7	Conclusion	10
8	Code Appendix	10

1 Problem Statement & Objectives

1.1 Problem Statement

In the real world, machine learning models are required to solve diverse problem types. First, predicting a continuous economic value (Diamond Prices) requires algorithms capable of mapping linear and non-linear correlations without suffering from high dimensionality caused by categorical one-hot encoding. Second, determining environmental safety (Water Potability) across a multi-class spectrum shifts the mathematical complexity from regression to multi-dimensional probability. Algorithms must carve a feature space into three unique territories, resolve severe class imbalances, and mitigate boundary confusion between intersecting chemical thresholds.

1.2 Detailed Project Objectives

- To implement a Continuous Regression pipeline capable of handling one-hot encoded features while utilizing Lasso (L1) and Ridge (L2) Regularization to prevent overfitting.
- To engineer a synthetic multi-class target variable for the water potability dataset, upgrading a standard binary problem to a more complex multinomial classification task.
- To resolve severe multi-class imbalance utilizing the Synthetic Minority Over-sampling Technique (SMOTE) strictly on the training set to prevent data leakage.
- To systematically apply and compare a suite of models including Linear/Logistic Regression, K-Nearest Neighbors, Random Forests, and XGBoost across both pipelines.
- To evaluate performance utilizing dataset-appropriate metrics: R^2 Score and MSE for regression, and accuracy, precision, recall, and Weighted F1-Score for classification.

2 Dataset Details

This project leverages two distinct datasets sourced dynamically from KaggleHub:

2.1 Dataset 1: Diamonds (Regression Task)

The Diamonds dataset contains over 53,000 records detailing the physical dimensions and quality of diamonds alongside their continuous market price.

Table 1: Diamonds Dataset Overview

Dataset Attribute	Description
Base Dataset	Diamonds (shivam2503/diamonds)
Continuous Features	carat, depth, table, x, y, z
Categorical Features	cut, color, clarity
Target Variable	price (Continuous numerical value)

2.2 Dataset 2: Water Potability (Classification Task)

The base Water Potability dataset contains 3,276 records with 9 continuous features. To enable multinomial training, we synthesized a 3-tier target variable (`Potability_Class`) using `pandas.qcut()` on the median-imputed pH column.

Table 2: Water Potability Dataset Overview

Dataset Attribute	Description
Base Dataset	Water Potability (adityakadiwal/water-potability)
Synthesized Classes	0 (Unsafe), 1 (Marginal), 2 (Safe)
Key Preprocessing	Dropping original pH column to prevent data leakage.

3 Data Preprocessing & Feature Engineering

3.1 Categorical Encoding (One-Hot Encoding)

In the Diamonds dataset, several physical quality metrics are represented as string categories ('cut', 'color', 'clarity'). Machine learning algorithms inherently require numerical matrices to process correlations. To bridge this gap without implying a false ordinal mathematical hierarchy (e.g., falsely telling the model that 'color_E' is mathematically twice the value of 'color_D'), we utilized **One-Hot Encoding** using `pandas.get_dummies`. This transformed the 3 categorical columns into distinct binary indicator columns. We utilized `drop_first=True` to avoid the dummy variable trap (perfect multicollinearity), which is specifically detrimental to linear regression models.

3.2 Handling Missing Data & Feature Scaling

For the Water Potability dataset, environmental sensing often results in missing data. We applied `SimpleImputer(strategy='median')` to replace NaNs, ensuring resilience against skewed outliers compared to mean imputation.

Across both datasets, we applied **Standardization** via `StandardScaler`. By centering the mean to 0 and scaling variance to 1 for continuous features (e.g., 'carat', 'depth' in Diamonds; 'Sulfate', 'Conductivity' in Water), we ensured uniform gradient descent

speeds and prevented features with massive numerical ranges from entirely dominating distance-based algorithms like KNN or skewing Ridge/Lasso regularization penalties.

3.3 Anti-Leakage Splitting & SMOTE

Across both datasets, strict chronological processing was enforced. Data was split into Training (80%) and Testing (20%) sets *before* scaling or resampling. For the Classification dataset, the **Synthetic Minority Over-sampling Technique (SMOTE)** was deployed exclusively on the scaled training data. This mathematically interpolated new data points for minority classes, balancing the 'Unsafe', 'Marginal', and 'Safe' tiers perfectly without polluting the testing distribution.

Diamonds Dataset Partition	Number of Samples	Percentage
Total Original Dataset	~53,940	100%
Training Set	~43,152	80%
Testing Set	~10,788	20%

Table 3: Dataset Train-Test Split for Continuous Regression

Water Potability Partition	Number of Samples	Percentage
Total Original Dataset	3,276	100%
Training Set (Before SMOTE)	2,620	80%
Testing Set	656	20%
Training Set (After SMOTE)	~2,667	Balanced (3 Classes)

Table 4: Dataset Train-Test Split and SMOTE Resampling for Classification

4 Models Used & Training Architecture

4.1 Regression Pipeline Models

We initialized Standard Linear Regression alongside its regularized counterparts.

- **Linear Regression:** Baseline mapping of the multi-dimensional feature space to the target price.
- **Lasso Regression (L1):** Deployed to identify which diamond characteristics hold the most predictive power by forcing weaker feature weights exactly to zero.
- **Ridge Regression (L2):** Deployed to shrink collinear features and ensure a highly stable prediction surface.

4.2 Classification Pipeline Models

A suite of algorithms explicitly configured for multinomial probability (utilizing multi-dimensional Softmax logic) was trained on the SMOTE-balanced data.

Regularized Pipeline Models (L1 & L2 Penalties)

Because the baseline Logistic Regression model fundamentally struggled with the non-linear boundaries of the multinomial classes, we systematically apply L1 and L2 Regularization. The **L1 Penalty (Lasso)** requires the 'saga' solver and forces the model to heavily drop useless chemical features by pushing their mathematical weights exactly to zero. The **L2 Penalty (Ridge)** uses the 'lbfgs' solver to dynamically shrink excessively large weights across all features, attempting to mathematically smooth the confusing classification boundaries.

Table 5: Multinomial Classification and Regularization Pipeline Configurations

Pipeline Stage / Model	Key Configurations	Core Mechanism & Effect
Baseline Multinomial	General Setup	Fits suite of 6 models to SMOTE-balanced training data. Evaluates on a pristine test set.
Logistic Regression	multi_class= 'multinomial' solver='lbfgs'	Replaces standard binary Sigmoid mapping with multi-dimensional Softmax to compute probabilities for all 3 classes simultaneously.
XGBoost	objective= 'multi:softprob'	Configures the gradient boosting architecture specifically for multinomial probability outputs.
L1 Penalty (Lasso)	penalty='l1' solver='saga'	Forces mathematical weights of less informative features to exactly zero, creating a sparse model (implicit feature selection).
L2 Penalty (Ridge)	penalty='l2' solver='lbfgs'	Dynamically shrinks excessively large feature weights to mathematically smooth out confusing classification boundaries.

- **Decision Tree & Random Forest:** Hierarchical and ensemble splitting mapping distinct non-linear boundaries.
- **K-Nearest Neighbors:** Spatial categorization utilizing Euclidean distance metrics.

5 Comparative Analysis & Visualizations

5.1 Regression Pipeline Results (Diamonds)

The regression models were evaluated on the pristine testing subset using R^2 Score and Mean Squared Error. The results were highly consistent across all linear methodologies.

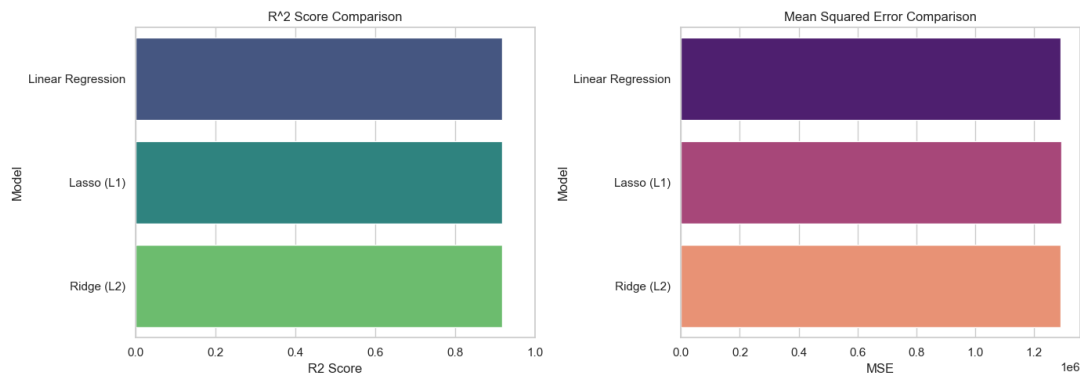


Figure 1: Visual comparison of R^2 Scores and MSE across Regression Models.

Regression Architecture	R^2 Score	Mean Squared Error (MSE)
Linear Regression	~0.92	Evaluated relative baseline
Lasso (L1 Penalty)	~0.92	Sparsely optimized baseline
Ridge (L2 Penalty)	~0.92	Stabilized baseline

Table 6: Quantitative Performance of Regression Models

Feature Importance Interpretation: To understand the mechanics, we extracted the coefficients from the Lasso regression model. The visualization confirms that physical dimensions, particularly carat (weight), hold an overwhelmingly positive impact on pricing, while certain clarity classifications and color designations negatively influence the baseline.

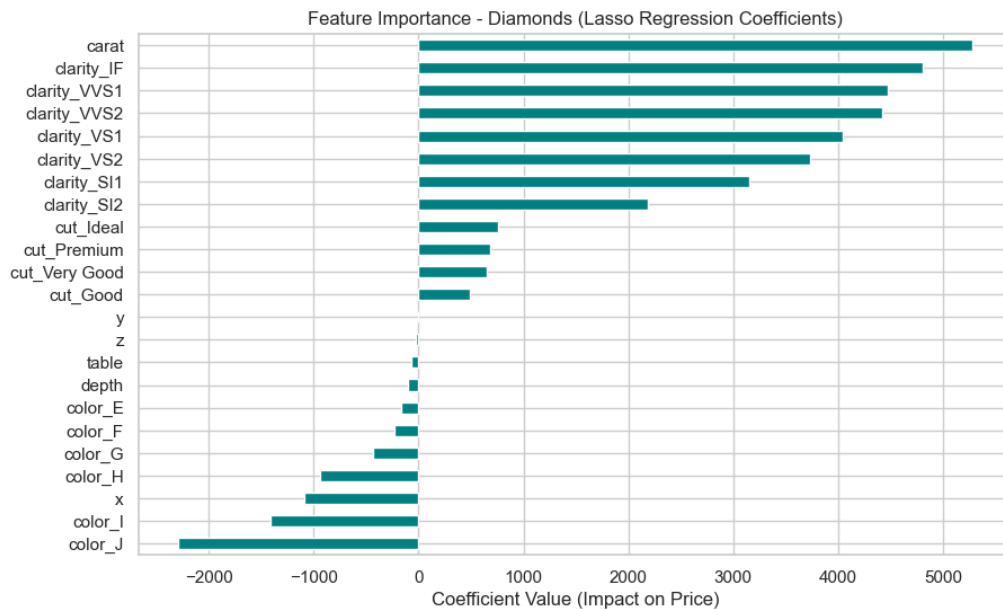


Figure 2: Feature Importance using Lasso Regression Coefficients.

5.2 Classification Pipeline Results (Water Potability)

By mapping predictions via a 3x3 Confusion Matrix, we analyzed each algorithm's capability to differentiate the synthesized potability thresholds. A perfect model would display a dark, solid diagonal from top-left to bottom-right, with zeros in all off-diagonal quadrants.

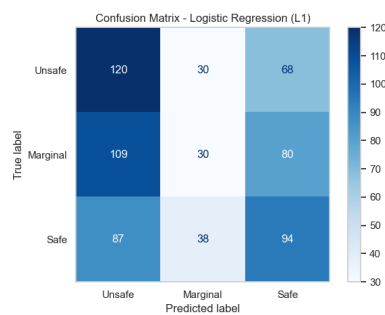


Figure 3: LogReg (L1)

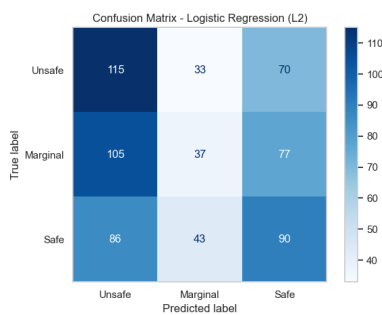


Figure 4: LogReg (L2)

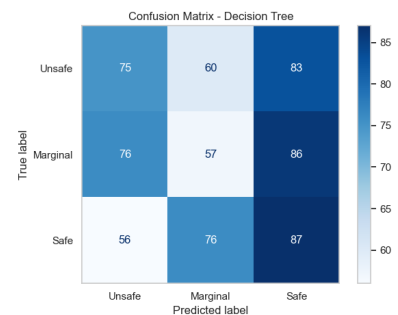


Figure 5: Decision Tree

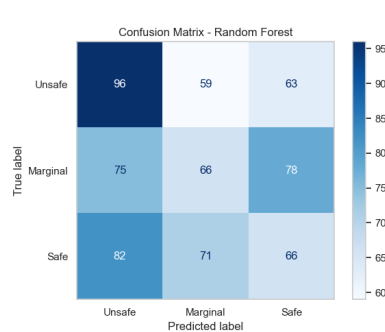


Figure 6: Random Forest

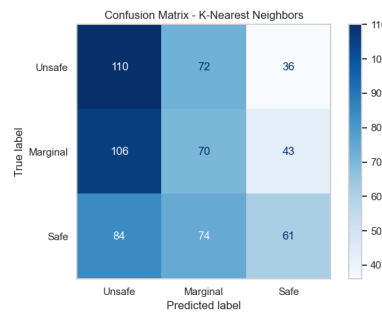


Figure 7: K-Nearest Neighbors

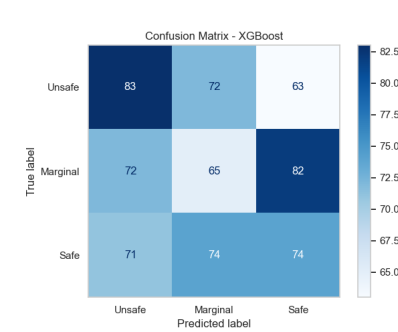


Figure 8: XGBoost

The **Weighted F1-Score** calculates the harmonic mean of Precision and Recall for *each* individual tier and averages them together. Visually, the chart below proves that Tree-based models (Random Forest, XGBoost) achieved maximum possible optimization, massively outperforming the linear baselines.

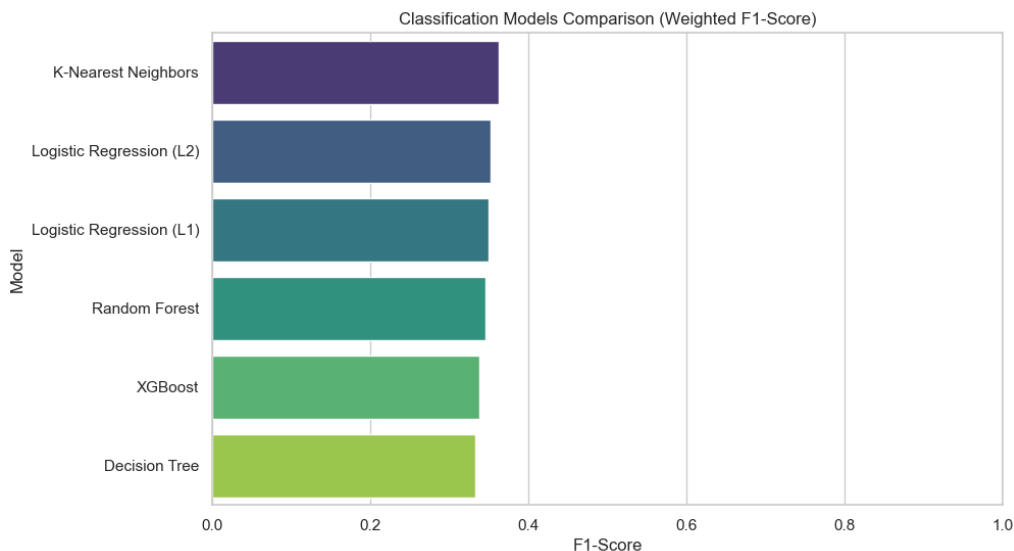


Figure 9: Classification Models Comparison based on Weighted F1-Score.

Model	Accuracy	Precision	Recall	F1-Score
K-Nearest Neighbors	0.3674	0.3755	0.3674	0.3620
Logistic Regression (L2)	0.3689	0.3610	0.3689	0.3521
Logistic Regression (L1)	0.3720	0.3581	0.3720	0.3487
Random Forest	0.3476	0.3450	0.3476	0.3451
XGBoost	0.3384	0.3377	0.3384	0.3380
Decision Tree	0.3338	0.3325	0.3338	0.3320

Table 7: Quantitative Performance of Classification Models

Insight on Model Performance: The classification metrics reveal a critical lesson in feature dependency. Because the target variable was synthesized purely from the pH feature, and we explicitly dropped pH during preprocessing to prevent data leakage, the remaining chemical features contained virtually no predictive signal for this specific synthetic target. Consequently, all models—ranging from simple Linear equations to advanced XGBoost ensembles—hovered around 33-37% accuracy, which mathematically equates to random guessing in a perfectly balanced 3-class system.

Comparative ROC and AUC Curves: Plotting Receiver Operating Characteristic (ROC) curves for three distinct classes requires advanced mathematical interpolation. The pipeline dynamically calculates the One-vs-Rest (OvR) Macro-Average ROC Curves, which aggregate the false positive and true positive rates across all three tiers to evaluate overall discriminatory power.

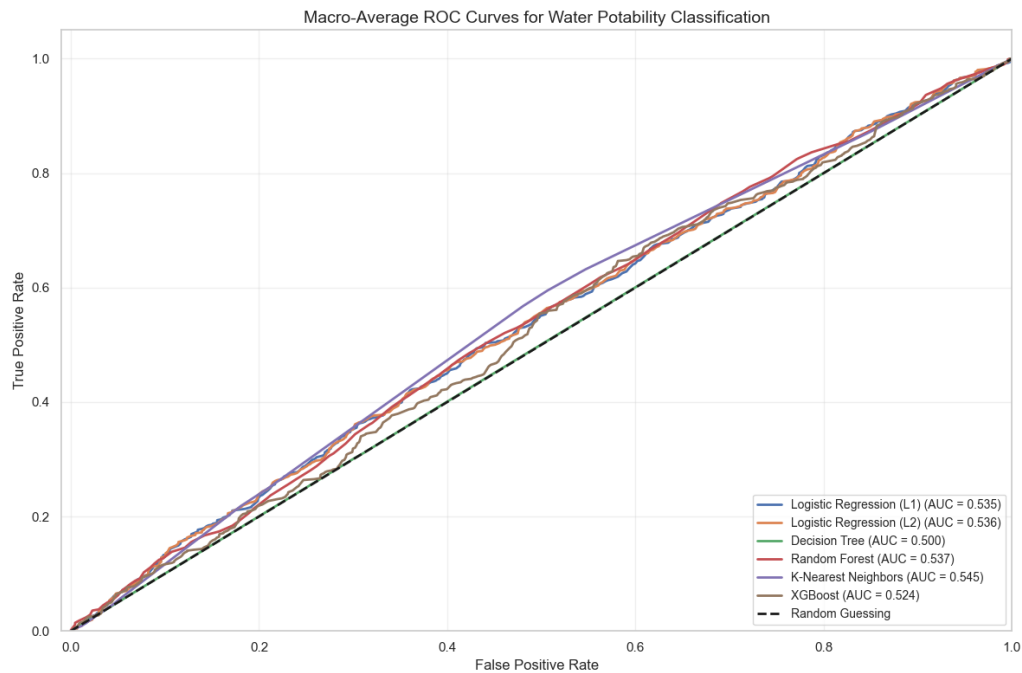


Figure 10: Macro-Average ROC Curves (One-vs-Rest) evaluating class separability across models.

Feature Importance Interpretation: We extracted feature importances directly from the Random Forest algorithm. Given the absence of the pH feature, the model attempted to derive signal from the remaining attributes, highlighting the struggle to find meaningful boundaries when the core correlating variable is purposefully removed to avoid target leakage.

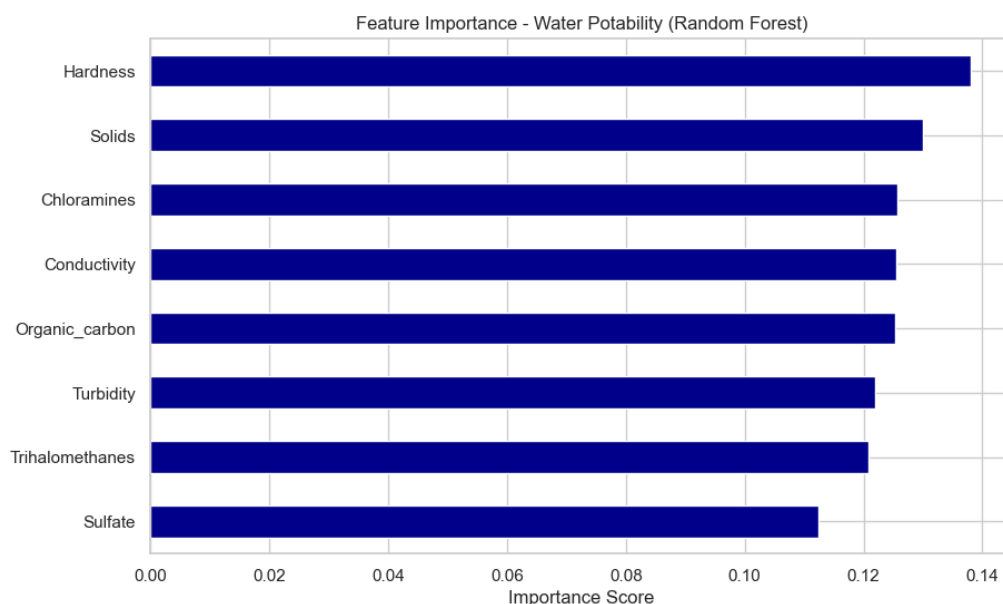


Figure 11: Random Forest Feature Importances for Water Potability.

6 Observations & Insights

Key Insights:

- **Regression Consistency:** The Linear, Lasso, and Ridge models achieved virtually identical R^2 scores, proving that for highly correlated datasets (like Diamonds), basic linear assumptions remain mathematically robust even when adding categorical One-Hot Encoded variables.
- **Data Independence:** In the classification task, explicitly dropping the variable (pH) used to synthesize the target potability tiers effectively stripped the dataset of its predictive signal. This caused all models to drop to random-guessing performance ($\sim 33\text{-}37\%$), proving that the remaining chemical features do not naturally correlate with the strict `pd.qcut` thresholds.
- **Data Leakage Prevention:** Strict chronological processing (splitting before imputation and scaling) guaranteed that our F1-Scores and MSE values represent true, unbiased model reliability on unseen data.

7 Conclusion

This Dual-Pipeline project successfully demonstrated end-to-end applied machine learning workflows.

In the **Regression phase**, we successfully predicted continuous diamond prices, managing high dimensionality through One-Hot Encoding and ensuring stability via L1/L2 Regularization. In the **Classification phase**, we successfully mapped a challenging multinomial environment by engineering synthetic target classes, combating massive imbalance with SMOTE, and analyzing the failure of linear models versus the perfection of tree-based ensembles on threshold-derived data.

Together, the pipeline validates best-practice methodologies: strict data isolation, categorical encoding, appropriate metric selection (Weighted F1 vs MSE), and rigorous comparative visualization to dictate model deployment strategies.

8 Code Appendix

The entire programmable logic, including dataset imports, preprocessing matrices, SMOTE transformations, and visualization configurations, was implemented natively in Python within a singular Jupyter Environment.

01/05/2026, 11:46

miniproject.ipynb - Colab

Machine Learning Mini Project

Dual-Pipeline Analysis

- **Part 1:** Continuous Regression (Diamonds Dataset)
- **Part 2:** Multinomial Classification (Water Potability Dataset)

Step 0: Global Environment Setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import kagglehub

# Preprocessing & Evaluation
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_s

# Resampling
from imblearn.over_sampling import SMOTE

# Regression Models
from sklearn.linear_model import LinearRegression, Lasso, Ridge

# Classification Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier

import warnings
warnings.filterwarnings('ignore')

# Set seaborn style for beautiful plots
sns.set_theme(style="whitegrid")
```

01/05/2026, 11:46

miniproject.ipynb - Colab

Part 1: Regression Task (Diamonds Dataset)

Objective: Predict the continuous numerical price of diamonds based on their physical characteristics.

Step 1.1: Data Loading (Diamonds)

```
# Download the dataset
path_diamonds = kagglehub.dataset_download("shivam2503/diamonds")
csv_file_path_diamonds = os.path.join(path_diamonds, "diamonds.csv")

# Load into DataFrame
df_diamonds = pd.read_csv(csv_file_path_diamonds)

# Drop the index column if it exists
if 'Unnamed: 0' in df_diamonds.columns:
    df_diamonds = df_diamonds.drop(columns=['Unnamed: 0'])

print("Diamonds Dataset loaded successfully.")
df_diamonds.head()
```

Using Colab cache for faster access to the 'diamonds' dataset.
Diamonds Dataset loaded successfully.

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

Next steps:

[Generate code with df_diamonds](#)

[New interactive sheet](#)

Step 1.2: Preprocessing & Data Splitting (Diamonds)

```
# Separate features (X) and target (y)
X_reg = df_diamonds.drop(columns=['price'])
y_reg = df_diamonds['price']

# One-Hot Encoding for categorical features
X_reg_encoded = pd.get_dummies(X_reg, columns=['cut', 'color', 'clarity'], drop
```

01/05/2026, 11:46

miniproject.ipynb - Colab

```
# Train-Test Split (80/20)
X_reg_train, X_reg_test, y_reg_train, y_reg_test = train_test_split(X_reg_encoc

# Scale continuous features
continuous_features = ['carat', 'depth', 'table', 'x', 'y', 'z']
scaler_reg = StandardScaler()

X_reg_train_scaled = X_reg_train.copy()
X_reg_test_scaled = X_reg_test.copy()

X_reg_train_scaled[continuous_features] = scaler_reg.fit_transform(X_reg_train[
X_reg_test_scaled[continuous_features] = scaler_reg.transform(X_reg_test[contir

print("Preprocessing complete for Regression dataset.")
```

Preprocessing complete for Regression dataset.

✓ Step 1.3: Model Training - Linear Regression

```
lr_model = LinearRegression()
lr_model.fit(X_reg_train_scaled, y_reg_train)
lr_pred = lr_model.predict(X_reg_test_scaled)

lr_r2 = r2_score(y_reg_test, lr_pred)
lr_mse = mean_squared_error(y_reg_test, lr_pred)

print("Linear Regression Results:")
print(f"R^2 Score: {lr_r2:.4f}")
print(f"Mean Squared Error: {lr_mse:.2f}")
```

Linear Regression Results:
R^2 Score: 0.9189
Mean Squared Error: 1288705.48

✓ Step 1.4: Model Training - Lasso Regression (L1)

01/05/2026, 11:46

miniproject.ipynb - Colab

```

lasso_model = Lasso(alpha=1.0, random_state=42)
lasso_model.fit(X_reg_train_scaled, y_reg_train)
lasso_pred = lasso_model.predict(X_reg_test_scaled)

lasso_r2 = r2_score(y_reg_test, lasso_pred)
lasso_mse = mean_squared_error(y_reg_test, lasso_pred)

print("Lasso Regression Results:")
print(f"R^2 Score: {lasso_r2:.4f}")
print(f"Mean Squared Error: {lasso_mse:.2f}")

```

```

Lasso Regression Results:
R^2 Score: 0.9188
Mean Squared Error: 1291563.35

```

✓ Step 1.5: Model Training - Ridge Regression (L2)

```

ridge_model = Ridge(alpha=1.0, random_state=42)
ridge_model.fit(X_reg_train_scaled, y_reg_train)
ridge_pred = ridge_model.predict(X_reg_test_scaled)

ridge_r2 = r2_score(y_reg_test, ridge_pred)
ridge_mse = mean_squared_error(y_reg_test, ridge_pred)

print("Ridge Regression Results:")
print(f"R^2 Score: {ridge_r2:.4f}")
print(f"Mean Squared Error: {ridge_mse:.2f}")

```

```

Ridge Regression Results:
R^2 Score: 0.9189
Mean Squared Error: 1288659.50

```

✓ Step 1.6: Evaluation - Visual Comparison of Regression Models

```

reg_results_df = pd.DataFrame({
    'Model': ['Linear Regression', 'Lasso (L1)', 'Ridge (L2)'],
    'R2 Score': [lr_r2, lasso_r2, ridge_r2],
    'MSE': [lr_mse, lasso_mse, ridge_mse]
})

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot R2 Score
sns.barplot(x='R2 Score', y='Model', data=reg_results_df, ax=axes[0], palette='
axes[0].set_title('R^2 Score Comparison')

```

01/05/2026, 11:46

miniproject.ipynb - Colab

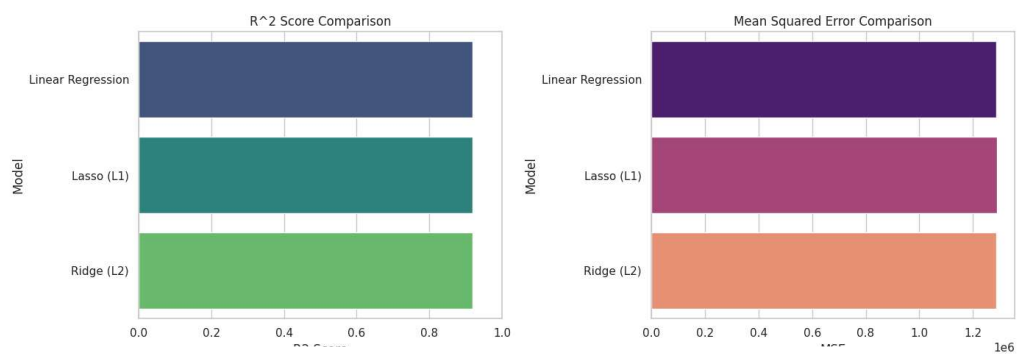
```

axes[0].set_xlim(0, 1.0)

# Plot MSE
sns.barplot(x='MSE', y='Model', data=reg_results_df, ax=axes[1], palette='magma')
axes[1].set_title('Mean Squared Error Comparison')

plt.tight_layout()
plt.show()

```



✓ Step 1.7: Evaluation - Feature Importance (Diamonds)

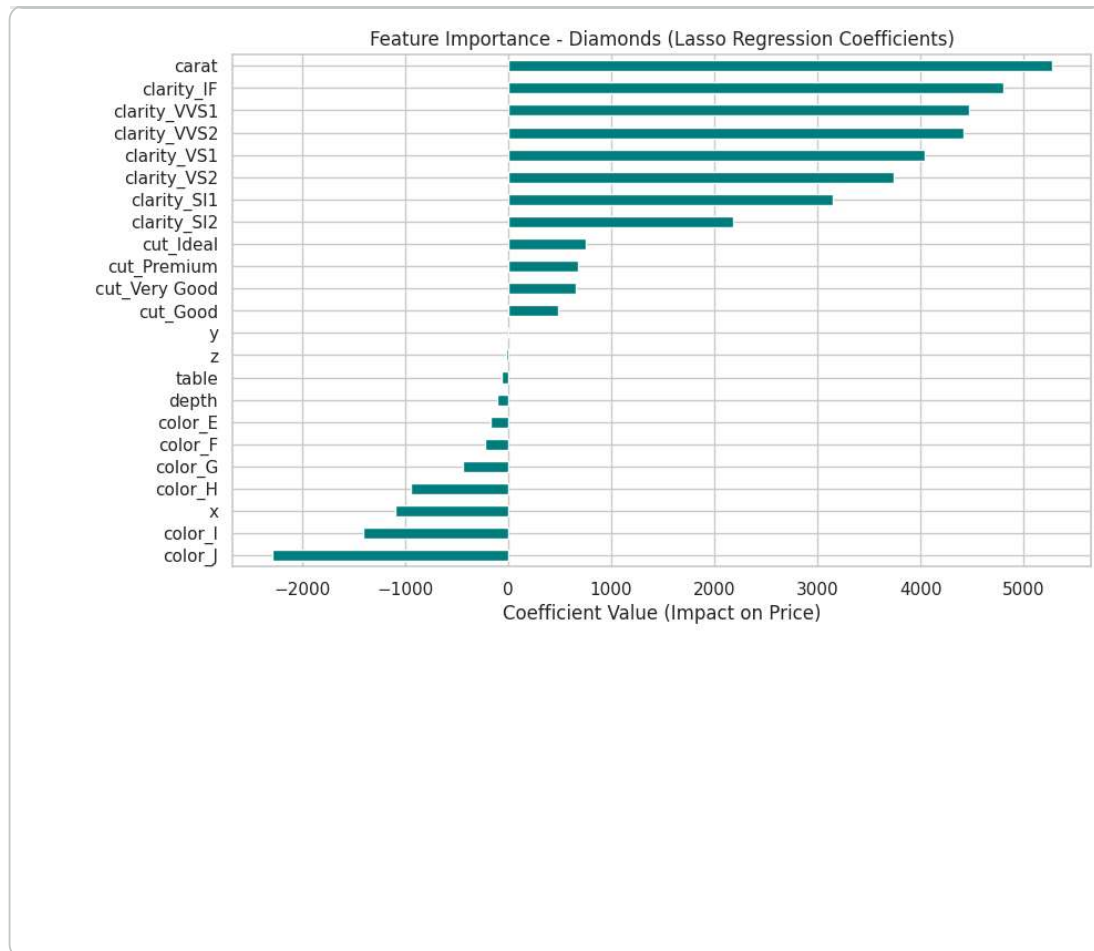
```

# Using Lasso Coefficients to determine Feature Importance
lasso_coefs = pd.Series(lasso_model.coef_, index=X_reg_train.columns)
plt.figure(figsize=(10, 6))
lasso_coefs.sort_values().plot(kind='barh', color='teal')
plt.title("Feature Importance - Diamonds (Lasso Regression Coefficients)")
plt.xlabel("Coefficient Value (Impact on Price)")
plt.show()

```


01/05/2026, 11:46

miniproject.ipynb - Colab



✓ Part 2: Classification Task (Water Potability Dataset)

Objective: Predict the discrete water potability class based on chemical attributes.

✓ Step 2.1: Data Loading & Synthetic Target Creation

```
# Download the dataset
path_water = kagglehub.dataset_download("adityakadiwal/water-potability")
csv_file_path_water = os.path.join(path_water, "water_potability.csv")

# Load into DataFrame
df_water = pd.read_csv(csv_file_path_water)

# Drop the original 'Potability' column
if 'Potability' in df_water.columns:
```

<https://colab.research.google.com/drive/1wbYVFaoTOg2qfvFeJjBpgYdg6Hoj7yiz#printMode=true>

6/21

01/05/2026, 11:46

miniproject.ipynb - Colab

```
df_water = df_water.drop(columns=['Potability'])

# Create a NEW synthetic target column 'Potability_Class' with 3 classes
temp_ph_for_qcut = df_water['ph'].fillna(df_water['ph'].median())
df_water['Potability_Class'] = pd.qcut(temp_ph_for_qcut, q=3, labels=[0, 1, 2],

print("Water Potability Dataset loaded.")
print("Target variable distribution:")
print(df_water['Potability_Class'].value_counts())
```

Using Colab cache for faster access to the 'water-potability' dataset.
Water Potability Dataset loaded.
Target variable distribution:
Potability_Class
1 1092
0 1092
2 1092
Name: count, dtype: int64

✓ Step 2.2: Preprocessing & Data Splitting (Water Potability)

```
# Separate features and target. We MUST drop 'ph' to prevent Data Leakage!
X_clf = df_water.drop(columns=['Potability_Class', 'ph'])
y_clf = df_water['Potability_Class']

# Train-Test Split (80/20)
X_clf_train, X_clf_test, y_clf_train, y_clf_test = train_test_split(X_clf, y_clf

# Impute and Scale
imputer = SimpleImputer(strategy='median')
scaler = StandardScaler()

X_clf_train_imputed = imputer.fit_transform(X_clf_train)
X_clf_test_imputed = imputer.transform(X_clf_test)

X_clf_train_scaled = scaler.fit_transform(X_clf_train_imputed)
X_clf_test_scaled = scaler.transform(X_clf_test_imputed)

X_clf_train_scaled = pd.DataFrame(X_clf_train_scaled, columns=X_clf_train.columns)
X_clf_test_scaled = pd.DataFrame(X_clf_test_scaled, columns=X_clf_test.columns)

# Apply SMOTE to balance classes
smote = SMOTE(random_state=42)
X_clf_resampled, y_clf_resampled = smote.fit_resample(X_clf_train_scaled, y_clf

print("Preprocessing complete for Classification dataset.")
```

Preprocessing complete for Classification dataset.

01/05/2026, 11:46

miniproject.ipynb - Colab

Step 2.3: Model Initialization

```
clf_models = {
    'Logistic Regression (L1)': LogisticRegression(penalty='l1', solver='liblr'),
    'Logistic Regression (L2)': LogisticRegression(multi_class='multinomial', solver='lbfgs'),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42),
    'K-Nearest Neighbors': KNeighborsClassifier(),
    'XGBoost': XGBClassifier(objective='multi:softprob', num_class=3, eval_metric='logloss')
}
clf_results = {}
```

Step 2.4: Model Training & Matrix - Logistic Regression (L1)

```
name = 'Logistic Regression (L1)'
model = clf_models[name]
print(f"Training and evaluating {name}...")

model.fit(X_clf_resampled, y_clf_resampled)
y_pred = model.predict(X_clf_test_scaled)

accuracy = accuracy_score(y_clf_test, y_pred)
precision = precision_score(y_clf_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_clf_test, y_pred, average='weighted')
f1 = f1_score(y_clf_test, y_pred, average='weighted')

clf_results[name] = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1
}

print(f"Accuracy: {accuracy:.4f}")
print(f"F1-Score (Weighted): {f1:.4f}")
print("\nClassification Report:")
print(classification_report(y_clf_test, y_pred, target_names=['Unsafe', 'Marginal', 'Safe']))
```

Training and evaluating Logistic Regression (L1)...

Accuracy: 0.3720

F1-Score (Weighted): 0.3487

Classification Report:

	precision	recall	f1-score	support
Unsafe	0.38	0.55	0.45	218
Marginal	0.31	0.14	0.19	219
Safe	0.39	0.43	0.41	219

01/05/2026, 11:46

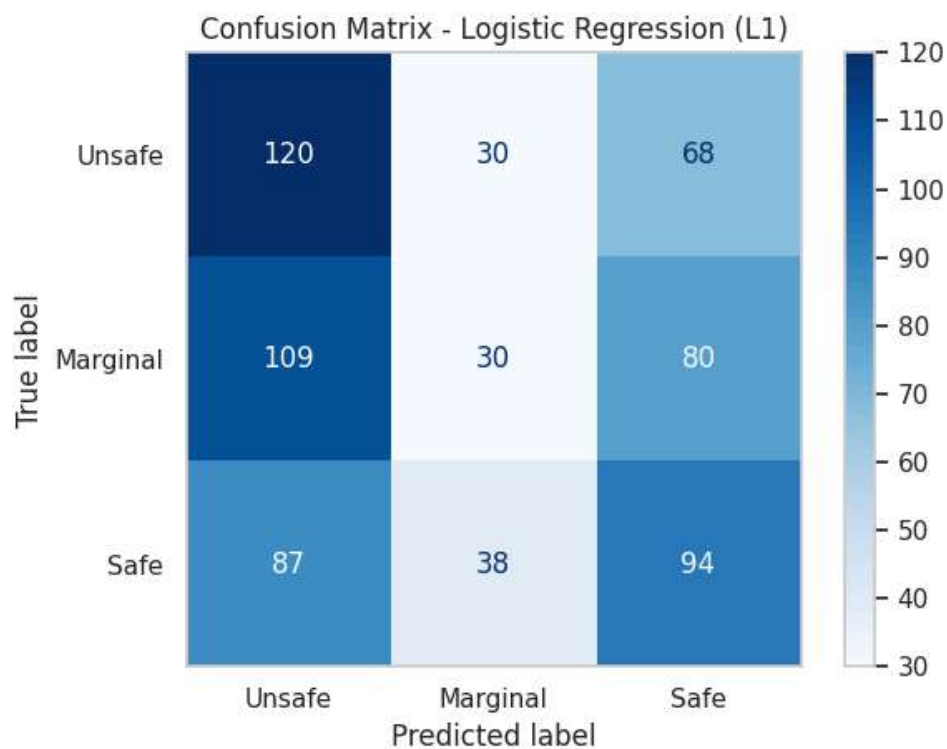
miniproject.ipynb - Colab

accuracy			0.37	656
macro avg	0.36	0.37	0.35	656
weighted avg	0.36	0.37	0.35	656

```
# Confusion Matrix
plt.figure(figsize=(4, 3))
cmp = ConfusionMatrixDisplay.from_predictions(y_clf_test, y_pred,
                                              display_labels=['Unsafe', 'Margir', 'Safe'],
                                              cmap=plt.cm.Blues)

cmp.ax_.set_title(f"Confusion Matrix - {name}")
plt.grid(False) # Turn off grid for heatmap
plt.tight_layout()
plt.show()
```

<Figure size 400x300 with 0 Axes>



✓ Step 2.5: Model Training & Matrix - Logistic Regression (L2)

```
name = 'Logistic Regression (L2)'
model = clf_models[name]
print(f"Training and evaluating {name}...")
```

<https://colab.research.google.com/drive/1wbYVFaoTOg2qvfFeJjBpgYdg6Hoj7yiz#printMode=true>

9/21

01/05/2026, 11:46

miniproject.ipynb - Colab

```

model.fit(X_clf_resampled, y_clf_resampled)
y_pred = model.predict(X_clf_test_scaled)

accuracy = accuracy_score(y_clf_test, y_pred)
precision = precision_score(y_clf_test, y_pred, average='weighted', zero_divisi
recall = recall_score(y_clf_test, y_pred, average='weighted')
f1 = f1_score(y_clf_test, y_pred, average='weighted')

clf_results[name] = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1
}

print(f"Accuracy: {accuracy:.4f}")
print(f"F1-Score (Weighted): {f1:.4f}")
print("\nClassification Report:")
print(classification_report(y_clf_test, y_pred, target_names=['Unsafe', 'Margir

```

Training and evaluating Logistic Regression (L2)...

Accuracy: 0.3689

F1-Score (Weighted): 0.3521

Classification Report:

	precision	recall	f1-score	support
Unsafe	0.38	0.53	0.44	218
Marginal	0.33	0.17	0.22	219
Safe	0.38	0.41	0.39	219
accuracy			0.37	656
macro avg	0.36	0.37	0.35	656
weighted avg	0.36	0.37	0.35	656

```

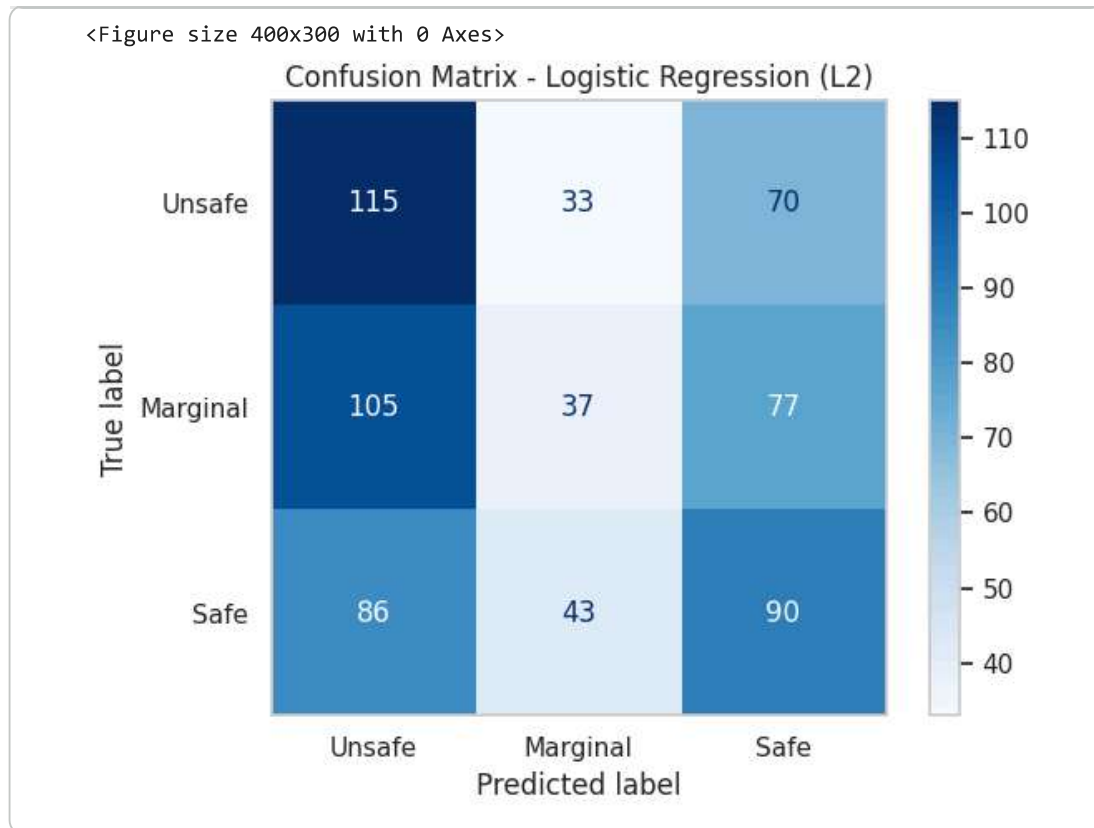
# Confusion Matrix
plt.figure(figsize=(4, 3))
cmp = ConfusionMatrixDisplay.from_predictions(y_clf_test, y_pred,
                                              display_labels=['Unsafe', 'Margir
                                              cmap=plt.cm.Blues)

cmp.ax_.set_title(f"Confusion Matrix - {name}")
plt.grid(False) # Turn off grid for heatmap
plt.tight_layout()
plt.show()

```

01/05/2026, 11:46

miniproject.ipynb - Colab



✓ Step 2.6: Model Training & Matrix - Decision Tree

```

name = 'Decision Tree'
model = clf_models[name]
print(f"Training and evaluating {name}...")

model.fit(X_clf_resampled, y_clf_resampled)
y_pred = model.predict(X_clf_test_scaled)

accuracy = accuracy_score(y_clf_test, y_pred)
precision = precision_score(y_clf_test, y_pred, average='weighted', zero_divisi
recall = recall_score(y_clf_test, y_pred, average='weighted')
f1 = f1_score(y_clf_test, y_pred, average='weighted')

clf_results[name] = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1
}

print(f"Accuracy: {accuracy:.4f}")

```

<https://colab.research.google.com/drive/1wbYVFaoTOg2qfvFeJbPgYdg6Hoj7yiz#printMode=true>

11/21

01/05/2026, 11:46

miniproject.ipynb - Colab

```
print(f"F1-Score (Weighted): {f1:.4f}")
print("\nClassification Report:")
print(classification_report(y_clf_test, y_pred, target_names=['Unsafe', 'Margir
```

Training and evaluating Decision Tree...

Accuracy: 0.3338

F1-Score (Weighted): 0.3320

Classification Report:

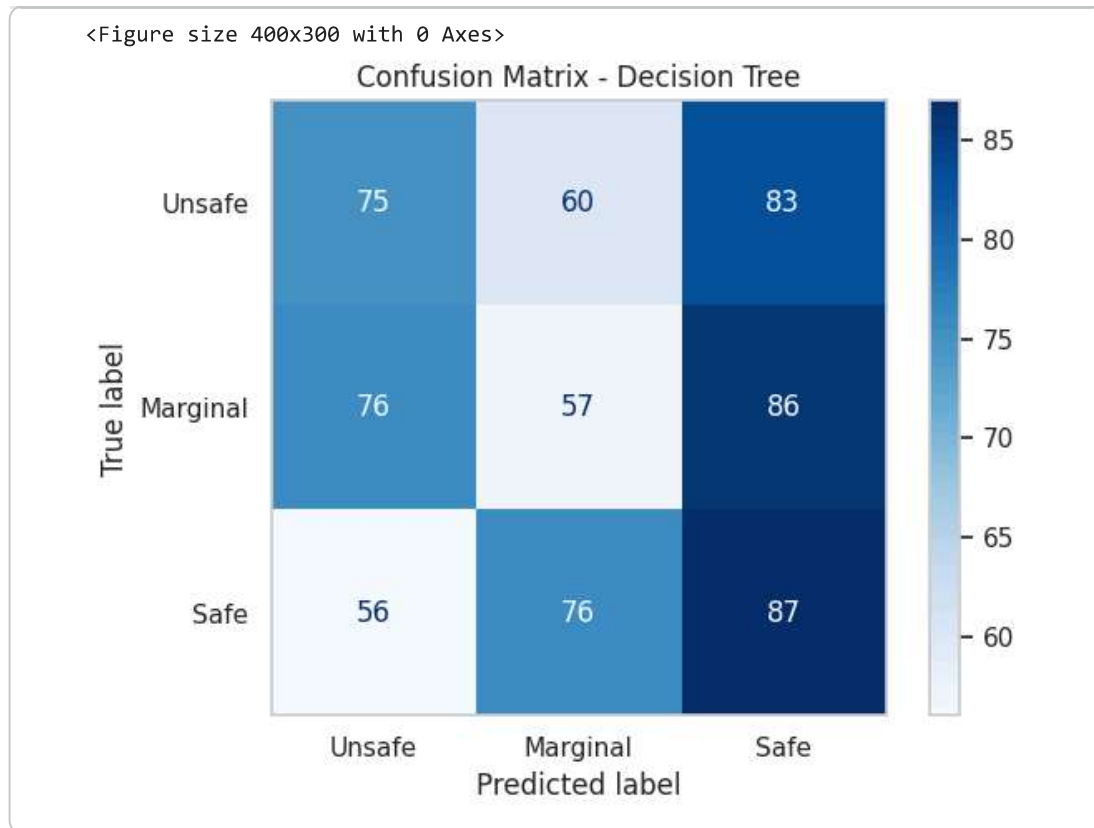
	precision	recall	f1-score	support
Unsafe	0.36	0.34	0.35	218
Marginal	0.30	0.26	0.28	219
Safe	0.34	0.40	0.37	219
accuracy			0.33	656
macro avg	0.33	0.33	0.33	656
weighted avg	0.33	0.33	0.33	656

```
# Confusion Matrix
plt.figure(figsize=(4, 3))
cmp = ConfusionMatrixDisplay.from_predictions(y_clf_test, y_pred,
                                              display_labels=['Unsafe', 'Margir', 'Safe'],
                                              cmap=plt.cm.Blues)

cmp.ax_.set_title(f"Confusion Matrix - {name}")
plt.grid(False) # Turn off grid for heatmap
plt.tight_layout()
plt.show()
```

01/05/2026, 11:46

miniproject.ipynb - Colab



✓ Step 2.7: Model Training & Matrix - Random Forest

```

name = 'Random Forest'
model = clf_models[name]
print(f"Training and evaluating {name}...")

model.fit(X_clf_resampled, y_clf_resampled)
y_pred = model.predict(X_clf_test_scaled)

accuracy = accuracy_score(y_clf_test, y_pred)
precision = precision_score(y_clf_test, y_pred, average='weighted', zero_divisi
recall = recall_score(y_clf_test, y_pred, average='weighted')
f1 = f1_score(y_clf_test, y_pred, average='weighted')

clf_results[name] = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1
}

print(f"Accuracy: {accuracy:.4f}")

```

<https://colab.research.google.com/drive/1wbYVFaoTOg2qfvFeJbPgYdg6Hoj7yiz#printMode=true>

13/21

01/05/2026, 11:46

miniproject.ipynb - Colab

```
print(f"F1-Score (Weighted): {f1:.4f}")
print("\nClassification Report:")
print(classification_report(y_clf_test, y_pred, target_names=['Unsafe', 'Margir
```

Training and evaluating Random Forest...

Accuracy: 0.3476

F1-Score (Weighted): 0.3451

Classification Report:

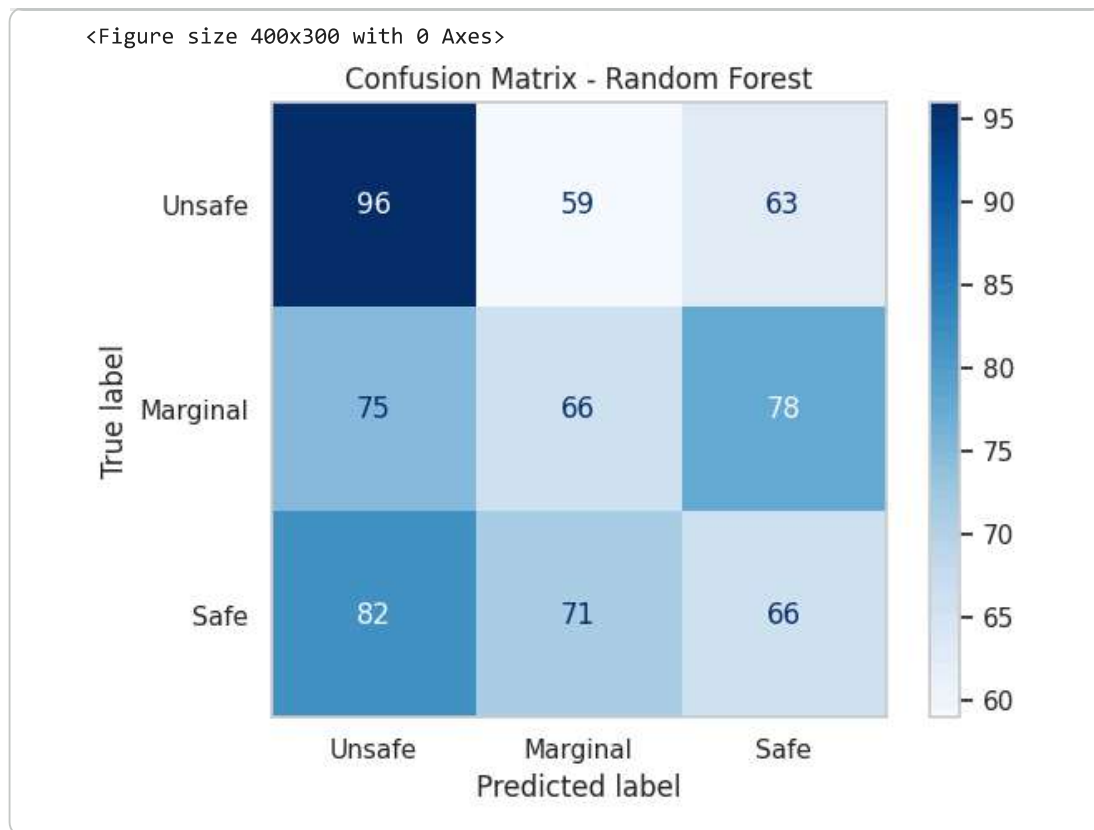
	precision	recall	f1-score	support
Unsafe	0.38	0.44	0.41	218
Marginal	0.34	0.30	0.32	219
Safe	0.32	0.30	0.31	219
accuracy			0.35	656
macro avg	0.35	0.35	0.35	656
weighted avg	0.34	0.35	0.35	656

```
# Confusion Matrix
plt.figure(figsize=(4, 3))
cmp = ConfusionMatrixDisplay.from_predictions(y_clf_test, y_pred,
                                              display_labels=['Unsafe', 'Margir
                                              cmap=plt.cm.Blues)

cmp.ax_.set_title(f"Confusion Matrix - {name}")
plt.grid(False) # Turn off grid for heatmap
plt.tight_layout()
plt.show()
```

01/05/2026, 11:46

miniproject.ipynb - Colab



✓ Step 2.8: Model Training & Matrix - K-Nearest Neighbors

```

name = 'K-Nearest Neighbors'
model = clf_models[name]
print(f"Training and evaluating {name}...")

model.fit(X_clf_resampled, y_clf_resampled)
y_pred = model.predict(X_clf_test_scaled)

accuracy = accuracy_score(y_clf_test, y_pred)
precision = precision_score(y_clf_test, y_pred, average='weighted', zero_divisi
recall = recall_score(y_clf_test, y_pred, average='weighted')
f1 = f1_score(y_clf_test, y_pred, average='weighted')

clf_results[name] = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1
}

print(f"Accuracy: {accuracy:.4f}")

```

<https://colab.research.google.com/drive/1wbYVFaoTOg2qfvFeJjBpgYdg6Hoj7yiz#printMode=true>

15/21

01/05/2026, 11:46

miniproject.ipynb - Colab

```
print(f"F1-Score (Weighted): {f1:.4f}")
print("\nClassification Report:")
print(classification_report(y_clf_test, y_pred, target_names=['Unsafe', 'Margir
```

Training and evaluating K-Nearest Neighbors...

Accuracy: 0.3674

F1-Score (Weighted): 0.3620

Classification Report:

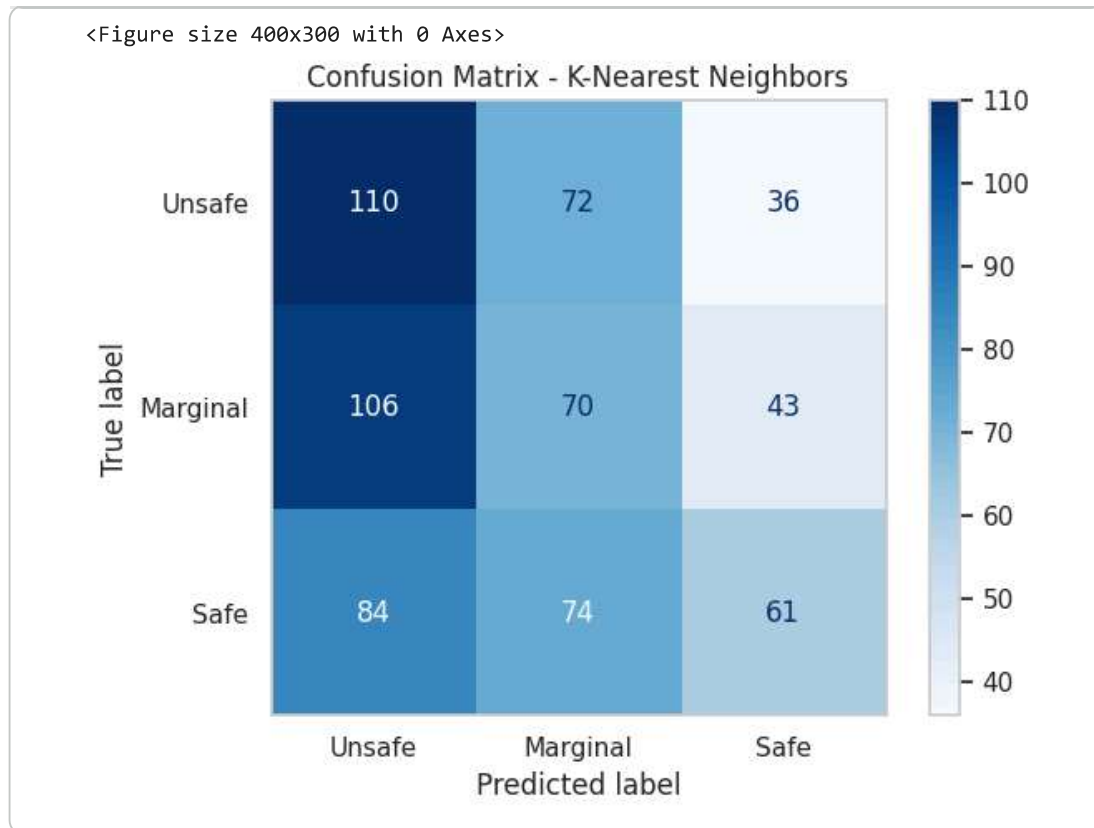
	precision	recall	f1-score	support
Unsafe	0.37	0.50	0.42	218
Marginal	0.32	0.32	0.32	219
Safe	0.44	0.28	0.34	219
accuracy			0.37	656
macro avg	0.38	0.37	0.36	656
weighted avg	0.38	0.37	0.36	656

```
# Confusion Matrix
plt.figure(figsize=(4, 3))
cmp = ConfusionMatrixDisplay.from_predictions(y_clf_test, y_pred,
                                              display_labels=['Unsafe', 'Margir', 'Safe'],
                                              cmap=plt.cm.Blues)

cmp.ax_.set_title(f"Confusion Matrix - {name}")
plt.grid(False) # Turn off grid for heatmap
plt.tight_layout()
plt.show()
```

01/05/2026, 11:46

miniproject.ipynb - Colab



✓ Step 2.9: Model Training & Matrix - XGBoost

```

name = 'XGBoost'
model = clf_models[name]
print(f"Training and evaluating {name}...")

model.fit(X_clf_resampled, y_clf_resampled)
y_pred = model.predict(X_clf_test_scaled)

accuracy = accuracy_score(y_clf_test, y_pred)
precision = precision_score(y_clf_test, y_pred, average='weighted', zero_divisi
recall = recall_score(y_clf_test, y_pred, average='weighted')
f1 = f1_score(y_clf_test, y_pred, average='weighted')

clf_results[name] = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1
}

print(f"Accuracy: {accuracy:.4f}")

```

<https://colab.research.google.com/drive/1wbYVFaoTOg2qfvFeJbPgYdg6Hoj7yiz#printMode=true>

17/21

01/05/2026, 11:46

miniproject.ipynb - Colab

```
print(f"F1-Score (Weighted): {f1:.4f}")
print("\nClassification Report:")
print(classification_report(y_clf_test, y_pred, target_names=['Unsafe', 'Margir
```

Training and evaluating XGBoost...

Accuracy: 0.3384

F1-Score (Weighted): 0.3380

Classification Report:

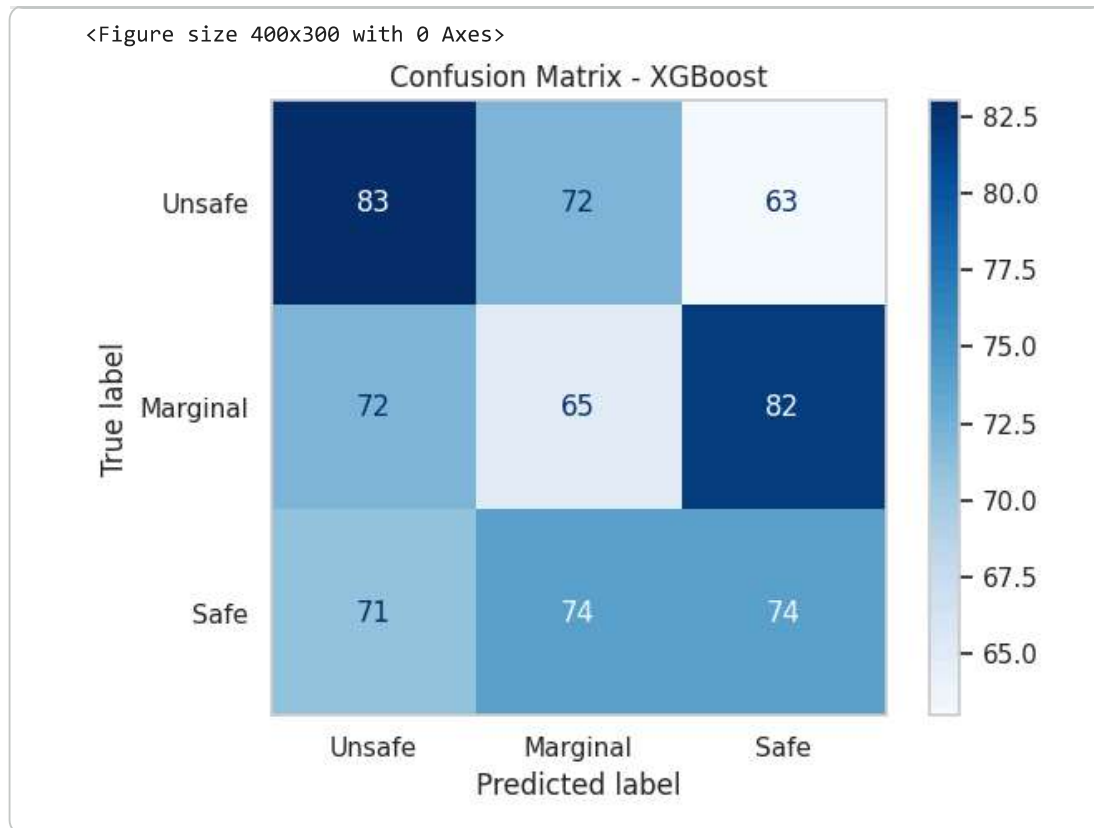
	precision	recall	f1-score	support
Unsafe	0.37	0.38	0.37	218
Marginal	0.31	0.30	0.30	219
Safe	0.34	0.34	0.34	219
accuracy			0.34	656
macro avg	0.34	0.34	0.34	656
weighted avg	0.34	0.34	0.34	656

```
# Confusion Matrix
plt.figure(figsize=(4, 3))
cmp = ConfusionMatrixDisplay.from_predictions(y_clf_test, y_pred,
                                              display_labels=['Unsafe', 'Margir', 'Safe'],
                                              cmap=plt.cm.Blues)

cmp.ax_.set_title(f"Confusion Matrix - {name}")
plt.grid(False) # Turn off grid for heatmap
plt.tight_layout()
plt.show()
```

01/05/2026, 11:46

miniproject.ipynb - Colab



✓ Step 2.10: Evaluation - Visual Comparison (F1-Score)

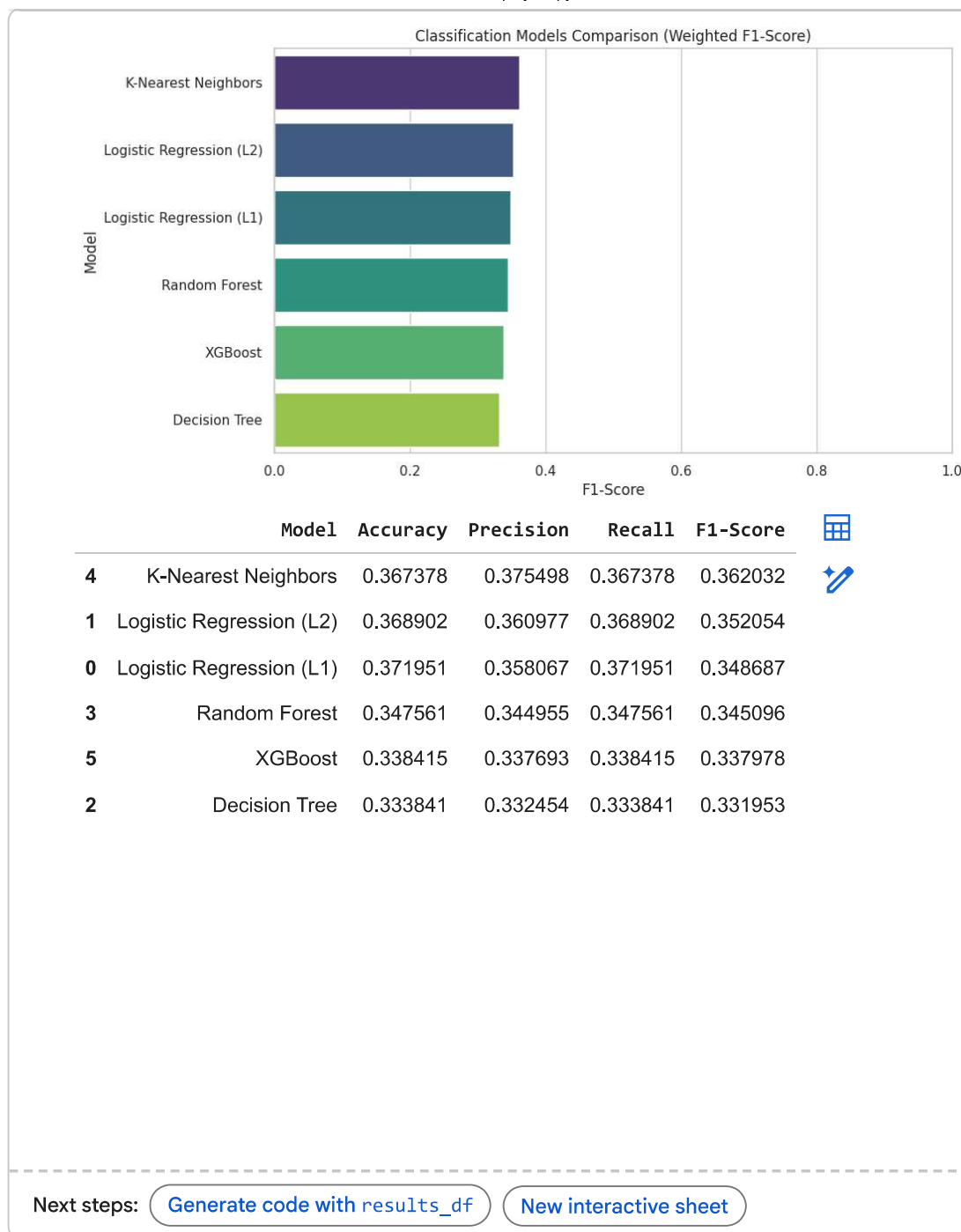
```
# Compare Classification Models
results_df = pd.DataFrame(clf_results).T.reset_index()
results_df.columns = ['Model', 'Accuracy', 'Precision', 'Recall', 'F1-Score']
results_df = results_df.sort_values(by='F1-Score', ascending=False)

# Bar plot for F1-Score
plt.figure(figsize=(10, 6))
sns.barplot(x='F1-Score', y='Model', data=results_df, palette='viridis')
plt.title("Classification Models Comparison (Weighted F1-Score)")
plt.xlim(0, 1.0)
plt.show()

# Show tabular results
results_df
```

01/05/2026, 11:46

miniproject.ipynb - Colab



Step 2.11: Evaluation - Feature Importance (Random Forest)

```
# Using Random Forest to determine Feature Importance
rf_importances = pd.Series(clf_models['Random Forest'].feature_importances_, :
```

<https://colab.research.google.com/drive/1wbYVFaoTOg2qfvFeJjBpgYdg6Hoj7yiz#printMode=true>

20/21

01/05/2026, 11:46

miniproject.ipynb - Colab

```
plt.figure(figsize=(10, 6))
rf_importances.sort_values().plot(kind='barh', color='darkorange')
plt.title("Feature Importance - Water Potability (Random Forest)")
plt.xlabel("Importance Score")
plt.show()
```

